Evolución de proyecto IonicReact

¿Por qué una ruta cambia pero la página no se renderiza si no devuelve `IonPage`?

Esta es una de las incidencias más frecuentes cuando se empieza a trabajar con Ionic React, especialmente si se viene de Angular o de React puro.

La situación suele ser la siguiente:

- La barra de navegación funciona.
- La URL cambia correctamente.
- React Router detecta la navegación.
- El navegador muestra la nueva ruta.
- Sin embargo la pantalla aparece vacía o el contenido no se renderiza correctamente.

El origen del problema suele estar en la forma en que Ionic gestiona internamente la navegación.

El error conceptual

Muchos desarrolladores procedentes de React intentan crear una página así:

```
function ClientsPage() {
  return (
        <div>
        <h1>Clientes</h1>
        </div>
  );
}
```

Desde el punto de vista de React puro esto es perfectamente válido.

Sin embargo, en Ionic React no siempre es suficiente.

React Router y Ionic Router no son exactamente lo mismo

Cuando utilizamos React Router puro, la arquitectura es:

BrowserRouter
↓
Route
↓
Component

El componente puede devolver cualquier estructura JSX.

Por ejemplo:

div
section
article
main

No existe ninguna restricción.

Arquitectura de Ionic React

Ionic añade una capa adicional:

IonReactRouter
↓
IonRouterOutlet
↓
IonPage
↓
Contenido

Aquí aparece el elemento clave:

IonPage

¿Qué es realmente IonPage?

`IonPage` no es simplemente un contenedor visual.

Internamente Ionic utiliza este componente para:

- Gestionar transiciones entre páginas.
- Gestionar animaciones.
- Controlar la navegación móvil.
- Administrar el ciclo de vida Ionic.
- Determinar qué página está activa.
- Gestionar la pila de navegación.

Desde la perspectiva del framework:

Una página Ionic debe estar encapsulada en un IonPage.

Qué espera Ionic internamente

Cuando una ruta es activada:

Route
↓
IonRouterOutlet
↓
Página

Ionic busca IonPage como elemento raíz.

Qué ocurre si no existe IonPage

- Renderizado incompleto.
- Página no visible.
- Problemas de altura.
- Problemas de scroll.
- Problemas de transición.
- Fallo en los ciclos de vida de Ionic.
- Pantallas aparentemente vacías.

La URL cambia. La navegación existe.

Pero Ionic no reconoce correctamente la página.

Estructura correcta

Toda página asociada a una ruta debería seguir esta estructura:

IonPage
  ↓
IonHeader
  ↓
IonToolbar
  ↓
IonContent

Arquitectura:

IonPage
  │
  ├── IonHeader
  │
  └── IonContent

Relación con los ciclos de vida Ionic

Otro problema muy frecuente.

Muchos usuarios intentan utilizar:

useEffect()

como en React puro.

Sin embargo Ionic incorpora ciclos de vida propios:

useIonViewWillEnter()

useIonViewDidEnter()

useIonViewWillLeave()

useIonViewDidLeave()

Estos ciclos dependen de que exista:

IonPage

Si la página no utiliza:

IonPage

pueden producirse comportamientos inesperados en la ejecución de estos hooks.

Diferencia con Angular Ionic

En Angular esta situación pasa desapercibida porque:

Ionic CLI

genera automáticamente páginas basadas en:

ion-page

y casi nunca se elimina.

Por tanto el alumno rara vez percibe que el componente existe.

En React estamos construyendo manualmente las páginas y es más fácil olvidarlo.

Arquitectura recomendada

IonReactRouter
↓
IonRouterOutlet
↓
ClientsPage
↓
IonPage
↓
IonContent
↓
ClientsTable

Buenas prácticas para AppClients

Pages

Deben devolver:

IonPage

Ejemplos:

- ClientsPage
- LoginPage
- NewClientPage
- EditClientPage

Components

No deben devolver:

IonPage

Ejemplos:

- ClientsTable
- ClientForm
- Navbar
- Toolbar
- AlertDialog

Ionic React: Render de Edit incorrecto

1. Desde `ClientsPage` se pulsa **Editar** sobre un cliente.
2. Se navega correctamente a `EditClientPage`.
3. El formulario aparece una única vez.
4. Se regresa a `ClientsPage`.
5. Se pulsa **Editar** sobre otro cliente.
6. Ahora aparecen dos formularios.
7. Se vuelve atrás.
8. Se pulsa nuevamente **Editar**.
9. Aparecen tres formularios.

Esto indica que existe un problema relacionado con:

Renderizados acumulados
Montajes repetidos
Suscripciones no liberadas
Duplicación de estado

y no con el Router en sí.

Primer diagnóstico

Cuando un componente aparece:

1 vez
2 veces
3 veces
4 veces...

normalmente NO estamos ante un problema de:

Supabase
PostgreSQL
Routing

sino de:

React Render
useEffect
Estado
JSX duplicado
Subscripciones

Causa 1. El formulario se está acumulando en un array

Muy habitual.

Supongamos:

```
setClients([
  ...clients,
  nuevoCliente
]);
```

o:

```
setForms([
  ...forms,
  nuevoFormulario
]);
```

Cada navegación añade un elemento nuevo.

Resultado:

Render 1 → 1 formulario
Render 2 → 2 formularios
Render 3 → 3 formularios

Cómo detectarlo

Inspeccionar la variable utilizada en el render:

```
console.log(client);
```

o:

```
console.log(editData);
```

Comprobar si realmente se está acumulando información.

Causa 2. useEffect provoca múltiples cargas

Ejemplo peligroso:

```
useEffect(() => {
  loadClient();
});
```

Observa que:

[]

no existe.

Consecuencia

Cada render provoca:

Render
↓
useEffect
↓
Carga datos
↓
setState
↓
Render
↓
useEffect
↓
...

Entramos en un ciclo de renderizados.

Forma correcta

```
useEffect(() => {

  loadClient();

}, []);
```

o:

```
useEffect(() => {

  loadClient();

}, [id]);
```

dependiendo del escenario.

Causa 3. Renderizado doble de React StrictMode

Muy frecuente en React moderno.

En:

main.tsx

o:

index.tsx

puede existir:

```
<React.StrictMode>

  <App/>

</React.StrictMode>
```

Efecto

En modo desarrollo React ejecuta algunos ciclos dos veces para detectar errores.

Por ejemplo:

Mount
Unmount
Mount

Esto puede producir:

- Consultas dobles
- Logs dobles
- Renderizados aparentes duplicados

StrictMode NO suele provocar:

1
2
3
4 formularios

acumulativos en cada navegación.

Por tanto normalmente no es el origen principal del problema descrito.

Causa 4. El formulario se añade al DOM en lugar de reemplazarse

Ejemplo conceptual:

```
return (

  <>
        {form}
  </>

);
```

más otra estructura que vuelve a insertar el mismo componente.

Resultado:

Componente renderizado dos veces

Comprobación

Buscar si aparece:

<EditForm />

más de una vez.

o:

<ClientForm />

más de una vez.

Causa 5. useIonViewWillEnter ejecutándose repetidamente

En Ionic React:

```
useIonViewWillEnter(() => {
  loadClient();
});
```

se ejecuta cada vez que la página entra en primer plano.

Esto es correcto.

Pero si dentro:

loadClient();

se hace algo como:

```
setClients(
 [...clients, data]
);
```

entonces cada vuelta acumula información.

Causa 6. El estado nunca se reinicia

Ejemplo:

```
const [
clients,
setClients
] = useState([]);
```

Carga:

```
setClients([
  ...clients,
  data
]);
```

Cada navegación:

Estado anterior
+
Nuevos datos

En un formulario de edición normalmente se debe hacer:

```
setClient(data);
```

NO:

```
setClient([
  ...client,
  data
]);
```

Causa 7. Problema típico con parámetros de ruta

Supongamos:

/edit-client/1

Después:

/edit-client/2

Si el efecto depende incorrectamente del id:

```
useEffect(() => {
  loadClient();
}, []);
```

el componente puede reutilizarse internamente por Ionic.

En este caso suele ser recomendable:

```
useEffect(() => {
  loadClient();
}, [id]);
```

Porque React detectará:

ID antiguo
↓
ID nuevo
↓
Recargar cliente

Causa 8. IonRouterOutlet mantiene páginas en memoria

Este punto es extremadamente importante.

Ionic no destruye automáticamente todas las páginas.

Para conseguir:

- Transiciones rápidas
- Historial móvil
- Animaciones nativas

Ionic mantiene una cache interna.

Arquitectura:

IonRouterOutlet
↓
ClientsPage
↓
EditClientPage

Las páginas pueden permanecer montadas.

Efecto

Código pensado para:

React Web puro

puede comportarse diferente en:

Ionic React

Por eso suele recomendarse:

useIonViewWillEnter()

en lugar de depender exclusivamente de:

useEffect()

cuando trabajamos con Ionic.

Solución recomendada para analizar la duplicidad de EditPage

Paso 1

Añadir:

```
console.log(
  'EditPage render'
);
```

dentro del componente.

Paso 2

Añadir:

```
console.log(
  'loadClient ejecutado'
);
```

dentro de la carga.

Paso 3

Mostrar:

```
console.log(client);
```

después de cargar.

Paso 4

Revisar si se está utilizando:

```
setClient(data);
```

o:

```
setClient([
  ...client,
  data
]);
```

Ordenar

Opción recomendada: ordenar en Supabase

En `ClientsService` modifica la consulta:

```
static async getClients() {

  const { data, error } =

        await supabase

        .from('clients')

        .select('*')

        .order('id', {
        ascending: true
        });

  if(error) {

        throw error;

  }

  return data || [];

}
```

Orden descendente

```
.order('id', {
  ascending: false
});
```

Alternativa: ordenar en React

También es posible ordenar después de obtener los datos.

```
const data =
  await ClientsService
          .getClients();

data.sort(
  (a, b) => a.id - b.id
);

setClients(data);
```

o descendente:

```
data.sort(
  (a, b) => b.id - a.id
);
```

¿Cuál es mejor?

Para un proyecto profesional:

Supabase
↓
Datos ordenados
↓
React
↓
Render

es la opción más recomendable porque:

- Reduce trabajo en el navegador.
- Escala mejor cuando existen miles de registros.
- Aprovecha los mecanismos de optimización de PostgreSQL.
- Mantiene la lógica de consulta en la capa de datos.

Paginación de Clientes

Vamos a implementar una paginación cliente que:

- Muestre 5 registros por página.
- Muestre botones de navegación.
- Permita moverse entre páginas.
- Actualice automáticamente la tabla.

Arquitectura de la solución

Actualmente:

Supabase
↓
ClientsPage
↓
Tabla HTML

Tras la paginación:

Supabase
↓
ClientsPage
↓
Lista completa
↓
Cálculo de página
↓
Tabla HTML

Paso 1. Crear estado para la página actual

Dentro de:

ClientsPage.tsx

Añadir:

```
const [currentPage, setCurrentPage] =
  useState(1);
```

Paso 2. Definir tamaño de página

Por ejemplo:

```
const recordsPerPage = 5;
```

Paso 3. Calcular índices

```
const lastIndex =
  currentPage * recordsPerPage;

const firstIndex =
  lastIndex - recordsPerPage;
```

Ejemplo

Página 1:

firstIndex = 0

lastIndex = 5

se muestran:

0
1
2
3
4

Página 2:

firstIndex = 5

lastIndex = 10

se muestran:

5
6
7
8
9
``

Paso 4. Obtener registros visibles

```
const currentClients =

  clients.slice(
        firstIndex,
        lastIndex
  );
```

Ahora:

currentClients

contiene únicamente los registros visibles.

Paso 5. Modificar el map()

Actualmente probablemente tengas:

clients.map(...)

Cambiar por:

currentClients.map(...)

Ejemplo:

```
<tbody>

 {

        currentClients.map(
        client => (

        <tr key={client.id}>

        <td>
        {client.id}
        </td>

        <td>
        {client.name}
        </td>

        <td>
        {client.email}
        </td>

        <td>
        {client.phone}
        </td>

        </tr>

        ))

 }

</tbody>
```

Paso 6. Calcular páginas totales

```
const totalPages =

  Math.ceil(

        clients.length

        /

        recordsPerPage

  );
```

Ejemplo

Si existen:

17 clientes

y:

5 por página

Resultado:

4 páginas

Paso 7. Generar botones de página

Crear:

```
const pageNumbers = [];

for(
  let i = 1;
  i <= totalPages;
  i++
) {

  pageNumbers.push(i);

}
```

Paso 8. Mostrar navegación inferior

Debajo de la tabla:

```
<div
className="d-flex justify-content-center mt-3">
 {
        pageNumbers.map(
        page => (
        <button
        key={page}
        className={
        page === currentPage
        ?
        "btn btn-primary mx-1"
        :
        "btn btn-outline-primary mx-1"
        }

        onClick={() =>
        setCurrentPage(page)
        }
        >
        {page}
        </button>
        ))
 }
</div>
```

Añadir botones Anterior y Siguiente

```
<div
className="d-flex justify-content-center mt-3">

 <button
        className=
        "btn btn-secondary mx-1"
        disabled={
        currentPage === 1
        }
        onClick={() =>
        setCurrentPage(
        currentPage - 1
        )
        }
 >Anterior</button>
 {
        pageNumbers.map(
        page => (
        <button
        key={page}
        className={
        page === currentPage
        ?
        "btn btn-primary mx-1"
        :
        "btn btn-outline-primary mx-1"
        }
        onClick={() =>
        setCurrentPage(
        page
        )
        }
        >
        {page}
        </button>
        ))
 }
```

```
 <button className="btn btn-secondary mx-1"

        disabled={
        currentPage ===
        totalPages
        }
        onClick={() =>
        setCurrentPage(
        currentPage + 1
        )
        }

 >

        Siguiente

  </button>

</div>
```

Implementación de Delete con actualización automática de la tabla

La funcionalidad debe cumplir los siguientes requisitos:

- El usuario selecciona un registro.
- Pulsa el botón Eliminar.
- Se solicita confirmación.
- Se elimina el registro en Supabase.
- La tabla se actualiza automáticamente.
- No es necesario recargar la página.

Arquitectura de la solución

La eliminación no debe realizarse directamente desde la tabla HTML.

La responsabilidad de cada capa será:

ClientsPage

↓

ClientsService

↓

Supabase

↓

PostgreSQL

Principio de Separación de Responsabilidades

Vamos a respetar el principio arquitectónico más importante del proyecto:

ClientsPage

Responsable de:

- Gestionar eventos.
- Gestionar estados.
- Coordinar acciones.
- Actualizar la interfaz.

ClientsService

Responsable de:

- Comunicarse con Supabase.
- Ejecutar operaciones CRUD.
- Gestionar errores de acceso a datos.

Supabase

Responsable de:

- Ejecutar realmente el DELETE sobre PostgreSQL.

Paso 1. Implementar deleteClient() en el servicio

Archivo:

src/services/ClientsService.ts

Añadimos el método de eliminación.

```
static async deleteClient(
  id: number
): Promise<void> {

  const { error } =

        await supabase

        .from('clients')

        .delete()

        .eq('id', id);

  if(error) {

        throw error;

  }

}
```

¿Por qué lanzamos el error?

```
if(error) {

  throw error;

}
```

Porque no queremos ocultar errores.

Si Supabase falla:

- Problemas de conexión.
- Permisos insuficientes.
- RLS mal configurado.
- Error interno.

la capa superior debe conocerlo.

Paso 2. Crear el botón de eliminación

Dentro de la tabla HTML.

Supongamos que actualmente tenemos:

```
<td>

 <Link
        to={/edit-client/${client.id}}
        className="btn btn-warning"
 >

        Editar

 </Link>

</td>
```

Vamos a ampliarlo.

```
<td>

 <Link
        to={/edit-client/${client.id}}
        className="btn btn-warning me-2"
 >

        Editar

 </Link>

 <button

        className="btn btn-danger"

        onClick={() =>
        deleteClient(client.id)
        }

 >
```

Eliminar

</button>

</td>

¿Por qué usamos una función flecha?

Incorrecto:

```
onClick={
  deleteClient(client.id)
}
```

Esto ejecutaría el borrado inmediatamente durante el renderizado.

Correcto:

```
onClick={() =>
  deleteClient(client.id)
}
```

Ahora la función solamente se ejecutará cuando el usuario pulse el botón.

Paso 3. Implementar deleteClient() en ClientsPage

Dentro del componente:

```
const deleteClient =
  async (id:number) => {

  };
```

Añadimos la lógica completa.

```
const deleteClient =
  async (
        id:number
  ) => {

        const confirmDelete =

        window.confirm(

        '¿Desea eliminar este cliente?'

        );

        if(!confirmDelete) {

        return;

        }

        try {

        await ClientsService
        .deleteClient(id);

        await loadClients();

        }
```

```
        catch(error) {

        console.error(error);

        alert(
        'Error eliminando cliente'
        );

        }

 };
```

Explicación técnica de la asincronía

La eliminación es una operación remota.

Arquitectura:

React
  ↓
Internet
  ↓
Supabase
  ↓
PostgreSQL

Por tanto:

deleteClient()

devuelve una Promise.

Necesitamos:

await

para esperar a que PostgreSQL termine el borrado.

¿Qué ocurriría sin await?

Ejemplo:

```
ClientsService
  .deleteClient(id);

loadClients();
```

Problema:

loadClients()

podría ejecutarse antes de terminar la eliminación.

Esto provoca inconsistencias.

Secuencia correcta

Eliminar

↓

Esperar respuesta

↓

Actualizar tabla

Paso 4. Actualización automática

Después del DELETE ejecutamos:

await loadClients();

¿Por qué recargar?

Porque React no conoce automáticamente que PostgreSQL ha cambiado.

React solamente conoce:

Estado

Si la base de datos cambia:

React sigue teniendo
el estado antiguo

hasta que volvamos a consultar.

Flujo completo

DELETE
↓
Supabase
↓
PostgreSQL
↓
Nueva consulta
↓
setClients()
↓
Nuevo render

Versión final recomendada

ClientsService

```
static async deleteClient(
  id:number
): Promise<void> {

  const { error } =

        await supabase

        .from('clients')

        .delete()

        .eq('id', id);

  if(error) {

        throw error;

  }

}
```

ClientsPage

```
const deleteClient =
  async (
        id:number
  ) => {

        const confirmDelete =

        window.confirm(
        '¿Desea eliminar este cliente?'
        );

        if(!confirmDelete) {

        return;

        }

        try {

        await ClientsService
        .deleteClient(id);

        await loadClients();

        }
        catch(error) {

        console.error(error);

        alert(
        'Error eliminando cliente'
        );

        }

};
```

Tabla HTML

```
<button

  className="btn btn-danger"

  onClick={() =>
          deleteClient(client.id)
  }

>

  Eliminar

</button>
```

Autenticación con Supabase y protección de Edit/Delete

Objetivo

Vamos a evolucionar la arquitectura para conseguir:

Usuario anónimo
↓
Puede consultar clientes

Usuario autenticado
↓
Puede editar clientes
Puede eliminar clientes

La seguridad se apoyará en:

Supabase Auth
JWT
Sesión
Policies RLS
React Router
Protected Routes

1. Arquitectura de autenticación

La arquitectura final será:

LoginPage
↓
AuthService
↓
Supabase Auth
↓
JWT
↓
Sesión Supabase
↓
Protected Routes
↓
EditClientPage

ClientsPage
↓
Mostrar/Ocultar Delete

2. Crear usuario en Supabase

Entrar en:

Supabase
→ Authentication
→ Users

Supabase almacena:

id
email
hash contraseña

Nunca almacena la contraseña en texto plano.

3. Concepto de sesión

Muchos usuarios vienen de aplicaciones clásicas.

Antes:

Login
↓
Servidor
↓
Session ID
↓
Memoria servidor

Hoy Supabase utiliza:

JWT

Flujo moderno

Email
Password
↓
Supabase Auth
↓
Access Token
Refresh Token
Session

Access Token

Contiene:

Identidad usuario
Permisos
Fecha expiración

Refresh Token

Permite renovar automáticamente el acceso.

Ventaja

La aplicación no necesita guardar sesiones manualmente.

Supabase las gestiona automáticamente.

4. Crear AuthService

Estructura

src/services

└── AuthService.ts

AuthService.ts

```
import { supabase }
from '../config/supabase';

export class AuthService {

  static async login(
    email:string,
    password:string
  ) {

    const {

      data,

      error

    } = await supabase.auth

      .signInWithPassword({

        email,
```

```
        password
      });
    if(error) {
      throw error;
    }
    return data;
  }
  static async logout() {
    await supabase.auth.signOut();
  }
  static async getSession() {
    const {
      data
    } = await supabase.auth
      .getSession();
    return data.session;
  }
  static async isAuthenticated() {
    const session =
      await this.getSession();
    return !!session;
  }
}
```

Explicación técnica

login()

Inicia sesión en Supabase.

---

getSession()

Obtiene la sesión actual.

---

isAuthenticated()

Devuelve:

true

si existe usuario autenticado.

5. Crear LoginPage

LoginPage.tsx

```
import {

  useState

} from 'react';

import {

  useHistory

} from 'react-router-dom';

import {

  AuthService

} from '../services/AuthService';

export default function LoginPage() {

  const history =
    useHistory();

  const [

    email,

    setEmail

  ] = useState('');

  const [

    password,

    setPassword

  ] = useState('');

  const login =
    async () => {

      try {
```

```
      await AuthService.login(

        email,

        password

      );

      history.push(
        '/clients'
      );

    }
    catch {

      alert(
        'Credenciales incorrectas'
      );

    }

  };

 return (

  <>
    {/* Formulario Login */}
  </>

 );

}
```

6. Añadir ruta Login

App.tsx

```
<Route

 path="/login"

 component={LoginPage}

 exact

/>
```

7. Mantener el usuario logado

La gran ventaja de Supabase es que:

Gestiona automáticamente la sesión.

Cuando el usuario recarga:

F5

la sesión sigue activa.

Comprobación:

```
const logged =

  await AuthService
    .isAuthenticated();
```

8. Crear ProtectedRoute vs Guards

Angular tiene:

Guards

React Router 5 no los tiene.

Debemos construirlos.

ProtectedRoute.tsx

```
import React from 'react';
import {
  Redirect,
  Route
} from 'react-router-dom';
import {
  AuthService
} from '../services/AuthService';
export default function ProtectedRoute({
  component: Component,
  ...rest
}: any) {
  return (
    <Route
      {...rest}
      render={(props) => {
        const session =
```

```
          localStorage.getItem(
            'supabase.auth.token'
          );

        return session

        ?

        <Component {...props}/>

        :

        <Redirect to="/login"/>

      }}

    />

  );

}
```

9. Proteger EditClientPage

Actualmente:

```
<Route
 path="/edit-client/:id"
 component={EditClientPage}
 exact
/>
```

Cambiar por:

```
<ProtectedRoute
 path="/edit-client/:id"
 component={EditClientPage}
 exact
/>
```

---

10. ¿Y Delete?

Delete NO es una ruta.

Delete es:

Un botón

Por tanto no se protege mediante Router.

Estrategia correcta

Paso 1

Detectar si hay sesión.

En:

ClientsPage.tsx

Estado:

```
const [
  isLogged,
  setIsLogged
] = useState(false);
```

Carga:

```
useEffect(() => {
  const checkAuth =
    async () => {
```

```
      const logged =

        await AuthService
          .isAuthenticated();

      setIsLogged(
        logged
      );

    };

  checkAuth();

}, []);
```

Paso 2

Mostrar botón sólo si existe sesión.

```
{

  isLogged &&

  <button

    className="btn btn-danger"

    onClick={() =>
      deleteClient(
        client.id
      )
    }

  >

    Eliminar

  </button>

}
```

---

11. Protección real en Supabase

Muy importante.

Ocultar botones:

NO es seguridad.

ProtectedRoute:

NO es seguridad.

¿Por qué?

Porque todo eso ocurre en:

Navegador

La seguridad real debe implementarse mediante:

Supabase Auth
+
RLS
+
Policies

Arquitectura final

React Router
↓
ProtectedRoute
↓
EditClientPage

ClientsPage
↓
isAuthenticated()
↓
Botón Delete

AuthService
↓
Supabase Auth
↓
JWT
↓
Session

RLS
↓
UPDATE

DELETE

Esta arquitectura replica el patrón profesional más habitual en aplicaciones React + Supabase: autenticación mediante JWT gestionados por Supabase, protección visual mediante React, protección de navegación mediante rutas protegidas y seguridad real garantizada por las políticas RLS de PostgreSQL.